



UNIVERSIDAD NACIONAL DE RÍO CUARTO

SISTEMAS OPERATIVOS

(Código 1965)

Taller 4: Threads y cambios de contexto

Departamento de Computación

Facultad de Ciencias Exactas, Físico-Químicas y Naturales

Alumno
Tomás Rodeghiero

Fecha de entrega: 17 de mayo de 2026

Índice

Estructura de la entrega	3
Ejercicio 1: cómo están implementados los threads y el cambio de contexto	3
Ejercicio 2: ¿por qué se guardan los registros s0-s11?	3
Ejercicio 3: ¿por qué no se guardan todos?	4
Ejercicio 4: traza pedida	4
Compilación y ejecución	5
Salida observada	5
Conclusión	5

Estructura de la entrega

- 04-context-switch/: paso original sin modificar.
- entrega/kernel.c: versión modificada.
- entrega/respuestas.txt: respuestas a los cuatro ejercicios.
- entrega/salida-kernel.txt: salida observada.
- informe-taller4-context-switch.pdf: este informe.
- README.md: descripción general.

Ejercicio 1: cómo están implementados los threads y el cambio de contexto

Un *thread* es su pila más su `sp` guardado. No hay todavía PCB ni scheduler: cada hilo se representa con dos variables globales en `kernel.c`, una para la pila y otra para acordarse del `sp` al ceder la CPU. El hilo principal (`kernel_main`) reusa la pila que le armó `boot` en `arch.s`, solo necesita el `sp` guardado.

Hay tres piezas que sostienen todo:

- `init_task_context(pc, sp)` (`arch.c`): apila 12 ceros (uno por cada `s0-s11`) y `pc` en el lugar de `ra`. La primera vez que se reanuda el hilo, el `ret` de `context_switch` saltará a su función de entrada.
- `context_switch(&cur, &next)` (`arch.s`): hace push de `ra` y `s0-s11` en la pila actual, intercambia `sp`, hace pop de los mismos registros en la pila nueva y vuelve. El `ret` usa el `ra` restaurado para saltar al punto donde ese hilo había cedido antes.
- `create_task` (`kernel.c`): envoltorio mínimo sobre `init_task_context`.

La idea principal es que `sp` define en qué hilo estamos. Cambiar de hilo es cambiar `sp` (más restaurar los registros que el ABI obliga a preservar).

Ejercicio 2: ¿por qué se guardan los registros `s0-s11`?

Porque la ABI de RISC-V los define como *callee-saved*: el llamador asume que mantienen su valor después de la llamada. El hilo que pierde la CPU puede tener variables vivas en esos registros; si no los guardáramos en su pila antes de cambiar de `sp`, el otro hilo los pisaría y al reanudar quedarían corruptos.

También se guarda `ra`, porque el `ret` final de `context_switch` lo usa para saltar a donde ese hilo había cedido la vez anterior. Sin `ra` no habría a dónde volver.

Ejercicio 3: ¿por qué no se guardan todos?

Los *caller-saved* (t0-t6, a0-a7) no hace falta. La ABI dice que el llamador no puede asumir que sobreviven a una llamada, así que el compilador ya derramó los que quería preservar antes de invocar a `context_switch`. Guardarlos sería trabajo redundante.

Los especiales tienen otro trato: `sp` se intercambia explícitamente (es la operación central), `tp` es del hart y no del hilo, `gp` es global y constante, y `zero` es siempre cero. Se guarda exactamente lo mínimo para que el ABI siga siendo correcto.

Ejercicio 4: traza pedida

Hay que producir:

```
main_thread -> task_a -> task_b -> task_a -> main_thread -> task_b
```

Son 5 cambios de contexto y 6 puntos de impresión. Agregué `task_b` con su pila y dispuse los `context_switch` así:

- `kernel_main` imprime, cede a `task_a`; al reanudar imprime y cede a `task_b`.
- `task_a` imprime, cede a `task_b`; al reanudar imprime y cede a `main`.
- `task_b` imprime, cede a `task_a`; al reanudar imprime y cede a `main` (cierre).

El `kernel.c` modificado queda así:

entrega/kernel.c

```

1  #include "klib.h"
2  #include "arch.h"
3
4  uint8  stack_task_a[PAGE_SIZE];
5  uint32* task_a_sp;
6
7  uint8  stack_task_b[PAGE_SIZE];
8  uint32* task_b_sp;
9
10 uint32* main_thread_sp;
11
12 uint32* create_task(uint32 pc, uint32 *sp)
13 {
14     return init_task_context(pc, sp);
15 }
16
17 void task_a(void)
18 {
19     printf("Task A on cpu %d\n", cpuid());
20     context_switch((uint32*) &task_a_sp, (uint32*) &task_b_sp);
21     printf("Task A again on cpu %d\n", cpuid());
22     context_switch((uint32*) &task_a_sp, (uint32*) &main_thread_sp);
23 }
24

```

```

25 void task_b(void)
26 {
27     printf("Task B on cpu %d\n", cpuid());
28     context_switch((uint32*) &task_b_sp, (uint32*) &task_a_sp);
29     printf("Task B again on cpu %d\n", cpuid());
30     context_switch((uint32*) &task_b_sp, (uint32*) &main_thread_sp);
31 }
32
33 void kernel_main(void) {
34     if (cpuid() == 0) {
35         task_a_sp = create_task((uint32) task_a,
36                                 (uint32*) (stack_task_a + PAGE_SIZE));
37         task_b_sp = create_task((uint32) task_b,
38                                 (uint32*) (stack_task_b + PAGE_SIZE));
39
40         printf("In kernel_main() thread\n");
41         context_switch((uint32*) &main_thread_sp, (uint32*) &task_a_sp);
42         printf("In kernel_main() thread again\n");
43         context_switch((uint32*) &main_thread_sp, (uint32*) &task_b_sp);
44     }
45     stop();
46 }

```

Compilación y ejecución

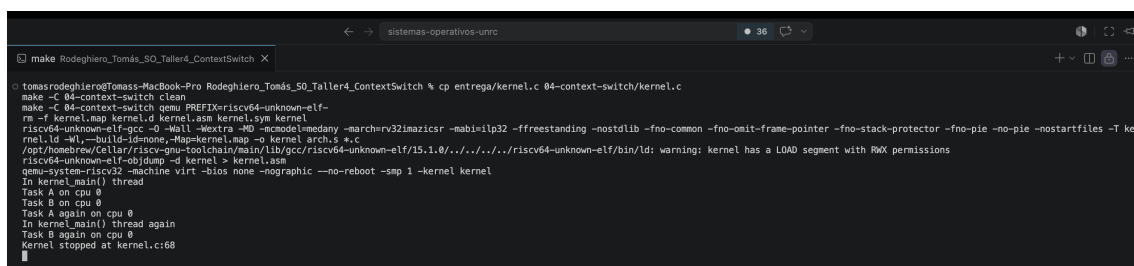
```

1 cp entrega/kernel.c 04-context-switch/kernel.c
2 make -C 04-context-switch clean
3 make -C 04-context-switch qemu PREFIX=riscv64-unknown-elf-

```

Para salir: Ctrl-A y después x.

Salida observada



```

make Rodeghiero_Tomás_SO_Taller4_ContextSwitch X
make -C 04-context-switch clean
make -C 04-context-switch qemu PREFIX=riscv64-unknown-elf-
rm -f kernel.map kernel.d kernel.asm kernel.sym kernel
riscv64-unknown-elf-gcc -D -Wall -Wextra -W0 -fcommon -fno-stack-protector -fno-pie -no-pie -nostartfiles -T ke
rnel.ld -Wl,--build-id=none,-Map=kernel.map -o kernel arch.s *.c
/opt/homebrew/Cellar/riscv-gnu-toolchain/main/lib/gcc/riscv64-unknown-elf/15.1.0/../../../../riscv64-unknown-elf/bin/ld: warning: kernel has a LOAD segment with RWX permissions
riscv64-unknown-elf-objdump -d kernel > kernel.asm
qemu-system-riscv32 -machine virt -bios none -nographic --no-reboot -smp 1 -kernel kernel
In kernel_main() thread
Task A on cpu 0
Task B on cpu 0
Task A again on cpu 0
In kernel_main() thread again
Task B again on cpu 0
Kernel stopped at kernel.c:68

```

Las seis líneas previas al `stop()` son, en orden, los puntos de impresión de la traza pedida.

Conclusión

En este taller pude ver el funcionamiento principal del cambio de contexto en pocas instrucciones: cada hilo es su pila más un `sp`, y cambiar de hilo es guardar los registros que el ABI exige, intercambiar `sp` y restaurar. Los ejercicios 2 y 3 justifican el conjunto: se guardan los *callee-saved* más `ra` y nada más, porque eso es lo mínimo que la ABI obliga a preservar.